自学的答案

3 级流水。有多种可能的答案，下面这个答案是一种比较合理的划分：

1. 取指令，读寄存器（300 ps）
2. ALU（200 ps）
3. 访问数据，写寄存器（300 ps）



时钟频率。图 4-27 给出的 5 级流水线的时钟周期是 200 ps，因此时钟频率是 1/200 ps，即 5GHz。3 级流水线中，流水级中最长的流水级延迟为 300 ps，因此时钟频率是 1/300 ps，即 3.33GHz。

寄存器读 / 写数据冒险。从画出的流水线中可以看出，3 级流水线里仍然存在读 / 写冒险。第 1 条指令在其第 3 级结束之前没有完成寄存器写入，而第 2 条指令再其第 2 级开始阶段就需要新的数据值。因为第 1 条指令的 ALU 运算结果在第 2 条指令低 2 级开始时已经准备好，所以图 4-8 中的旁路解决方案在该 3 级流水线中能够很好的工作。

取数 – 使用型数据冒险。与 4.8 节中的情况类似，即使对于 3 级流水线，取数 – 使用型数据冒险必须阻塞一个时钟周期。在 load 指令的第 3 级之前，数据没有准备好，而后续指令在自己的第 2 级开始时就要使用数据。

控制冒险。该类冒险在 3 级流水线中依然存在。可以使用与 4.9 节相同的优化方法，在 ALU 级之前计算分支地址，并对寄存器进行相等比较（与图 4-62 中所示类似）。地址计算在取下一条指令之前进行，因此提前的分支逻辑可以在没有流水线开销的情况下解决控制冒险。

CPI。3 级流水线的 CPI 将减少（变好），主要原因如下：

- 因为时钟周期加长，访问 DRAM 所需的时钟周期数量减少，这将在 cache 缺失（见第 5 章）时影响 CPI。
- 分支指令只需一个时钟周期，而在 5 级流水线中，用于加速分支指令的任何软件和硬件策略都在某些情况下失效，从而增加有效的 CPI。
- 5 级流水线中 ALU 可能支持多于一个周期的复杂操作，例如整数乘法或除法操作。3 级流水线中，由于时钟周期比 ALU 的运算时间长，因此对于这些复杂操作，需要的时钟周期数量少于 5 级流水线。

4.19　练习题

4.1　考虑如下指令：

指令：`and rd, rs1, rs2`

解释：`Reg[rd]=Reg[rs1] and Reg[rs2]`

4.1.1　[5] <4.3> 对上述指令，图 4-10 中的控制单元将产生哪些控制信号？

4.1.2　[5] <4.3> 对上述指令，将用到哪些功能单元？

4.1.3　[10] <4.3> 哪些功能单元会产生输出，但输出不会被以上指令用到？对以上指令，哪些功能单元不产生任何输出？

4.2　[10] <4.4> 解释图 4-18 中的每个“无关项”。

4.3 考虑下面的指令组合：

R-type	I-type (non-lw)	Load	Store	Branch	Jump
24%	28%	25%	10%	11%	2%

4.3.1 [5]<4.4> 多少指令会用到数据存储器？

4.3.2 [5]<4.4> 多少指令会用到指令存储器？

4.3.3 [5]<4.4> 多少指令会用到符号扩展？

4.3.4 [5]<4.4> 不需要符号扩展输出的时钟周期内，符号扩展部件在做什么？

4.4 在制造硅芯片时，材料（例如，硅）的缺陷和制造错误会导致电路失效。一种非常常见的缺陷是信号线损坏，使得信号线总是为逻辑 0，称为“固定 0”（stuck-at-0）故障。

4.4.1 [5]<4.4> 如果 MemToReg 信号线固定为 0，哪些指令将不能正常操作？

4.4.2 [5]<4.4> 如果 ALUSrc 信号线固定为 0，哪些指令将不能正常操作？

4.5 本题将详细考察一条指令在单周期数据通路中如何执行。本题中的问题都是在一个时钟周期内，其中处理器取入的指令字为：0x00c6ba23。

4.5.1 [10]<4.4> 对该指令，ALU 控制单元的输入值是什么？

4.5.2 [10]<4.4> 该指令执行后，新的 PC 地址是多少？高亮标出该值决定的路径。

4.5.3 [10]<4.4> 给出指令执行过程中，每个多路选择器的输入和输出值。列出寄存器 Reg[xn] 的输出值。

4.5.4 [10]<4.4>ALU 和两个加法单元的输入值是什么？

4.5.5 [10]<4.4> 寄存器单元的所有输入值是什么？

4.6 4.4 节没有讨论像 addi 或 andi 一类的 I 型指令。

4.6.1 [5]<4.4> 如果在图 4-21 中的 CPU 中增加 I 型指令，需要增加哪些逻辑块？在图 4-21 中增加必需的逻辑块，并解释增加这些逻辑块的目的。

4.6.2 [10]<4.4> 列出控制单元为 addi 指令产生的控制信号值，并解释所有“无关”控制信号的原因。

4.7 本题假定在实现一个处理器的数据通路时，逻辑模块的延时如下：

I-Mem / D-Mem	Register File	Mux	ALU	Adder	Single gate	Register Read	Register Setup	Sign extend	Control
250ps	150ps	25ps	200ps	150ps	5ps	30ps	20ps	50ps	50ps

“寄存器读”是指时钟上升沿到来后寄存器值出现在输出端所需的时间。这个值只适用于 PC。“寄存器建立”时间是指时钟上升沿到来之前，寄存器输入数据必需保持稳定的时间。这个值同时适用于 PC 和寄存器堆。

4.7.1 [5]<4.4>R 型指令的延迟是多少？（即 R 型指令正确执行需要多长的时钟周期？）

4.7.2 [10]<4.4>lw 指令的延迟是多少？（仔细检查你的答案。许多学生在关键路径上放置了额外的多路选择器）

4.7.3 [10]<4.4>sw 指令的延迟是多少？（仔细检查你的答案。许多学生在关键路径上放置了额外的多路选择器）

4.7.4 [5]<4.4>beq 指令的延迟是多少？

4.7.5 [5]<4.4> 一条算术指令、逻辑指令或 I 型的移位（非 load）指令的延迟是多少？

4.7.6 [5]<4.4> 该 CPU 的最小时钟周期是多少？

4.8 [10]<4.4> 假设你能够设计一个 CPU，其每条指令的时钟周期都不同。对于下面的指令使用频度，该新 CPU 相对于图 4-21 中的 CPU 的性能加速比是多少？

R-type/I-type (non-lw)	lw	sw	beq
52%	25%	11%	12%

4.9 考虑图 4-21 中所示的 CPU，如果在其中增加一个乘法器后，ALU 的延迟增加了 300 ps，但指令数减少了 5%（因为不再需要模拟乘法指令）

4.9.1 [5] <4.4> 改进前后的时钟周期各是多少？

4.9.2 [10] <4.4> 改进后获得的性能加速比是多少？

4.9.3 [10] <4.4> 在仍能改进性能的前提下，ALU 最慢（即最长延迟）是多少？

4.10 处理器设计师改进处理器的数据通路时，通常依赖于开销 / 性能之间的折中。在下面三个问题中，假设从图 4-21 给出的数据通路开始，采用和练习题 4.7 中相同的延迟，开销如下表所示：

I-Mem	Register File	Mux	ALU	Adder	D-Mem	Single Register	Sign extend	Sign gate	Control
1000	200	10	100	30	2000	5	100	1	500

假设将通用寄存器的数量从 32 个增加到 64 个时，将把 `lw` 和 `sw` 指令的数量减少 12%，但将寄存器堆的延迟从 150 ps 增加到 160 ps，开销从 200 增加到 400.（采用练习题 4.8 给出的指令使用频度，忽略练习题 2.18 中讨论的指令集体系结构的其他影响。）

4.10.1 [5] <4.4> 改进后的性能加速比能达到多少？

4.10.2 [10] <4.4> 比较性能的变化和开销的变化。

4.10.3 [10] <4.4> 结合计算出的开销 / 性能比，分别描述哪种情况下可以增加更多的寄存器，哪种情况下增加寄存器数量是没有意义的。

4.11 考察将指令 `lwi.d rd, rs1, rs2`（递增取数）添加到 MIPS 指令集中的难度。

解释：`Reg[rd]=Mem[Reg[rs1]+Reg[rs2]]`

4.11.1 [5] <4.4> 该指令需要哪些新的功能块（如果有）？

4.11.2 [5] <4.4> 哪些已经存在的功能块（如果有）需要修改（如果有）？

4.11.3 [5] <4.4> 该指令需要哪些新的数据通路（如果有）？

4.11.4 [5] <4.4> 需要控制器中哪些新的控制信号（如果有）来支持该指令？

4.12 考察将指令 `swap rs, st` 添加到 MIPS 指令集中的难度。

解释：`Reg[rt]= Reg[rs], Reg[rs]= Reg[rt]`

4.12.1 [5] <4.4> 该指令需要哪些新的功能块（如果有）？

4.12.2 [10] <4.4> 哪些已经存在的功能块（如果有）需要修改（如果有）？

4.12.3 [5] <4.4> 该指令需要哪些新的数据通路（如果有）？

4.12.4 [5] <4.4> 需要控制器中哪些新的控制信号（如果有）来支持该指令？

4.12.5 [5] <4.4> 修改图 4-21，给出这条新指令的实现方式。

4.13 考察将指令 `ss rt, rs, imm`（存和）添加到 MIPS 指令集中的难度。

解释：`MEM[Reg[rt]]= Reg[rs]+immediate`

4.13.1 [5] <4.4> 该指令需要哪些新的功能块（如果有）？

4.13.2 [10] <4.4> 哪些已经存在的功能块（如果有）需要修改（如果有）？

4.13.3 [5] <4.4> 该指令需要哪些新的数据通路（如果有）？

4.13.4 [5] <4.4> 需要控制器中哪些新的控制信号（如果有）来支持该指令？

4.13.5 [5] <4.4> 修改图 4-21，给出这条新指令的实现方式。

4.14 [5] <4.4> 哪条指令（如果有）使得立即数产生（Imm Gen）模块处于关键路径上？

4.15 lw 是 4.4 节 CPU 中延迟最长的指令。如果修改 lw 和 sw 指令，去掉偏移量（offset）计算（例如，在使用 lw 或 sw 指令之前必须将地址计算出来并存放在 rs 中），那么就没有指令同时使用 ALU 和数据存储器。这种修改可以减少时钟周期。然而，这也增加了指令数，因为许多 lw 和 sw 指令需要换成 lw/add 或 sw/add 的指令组合。

4.15.1 [5]<4.4> 新的时钟周期是多少？

4.15.2 [10]<4.4> 有练习题 4.7 中的混合指令构成的程序在这个新 CPU 上运行更快还是更慢？快或慢多少？（为了简化，假设每条 lw 和 sw 指令分别由连续的两条指令替代。）

4.15.3 [5]<4.4> 影响程序在新 CPU 上的运行速度的主要因素是什么？

4.15.4 [5]<4.4> 原来的 CPU（图 4-21 所示）与新的 CPU 相比，哪个的整体设计更好？

4.16 本练习讨论流水线对处理器时钟周期的影响。假设数据通路中各流水级的延迟如下：

IF	ID	EX	MEM	WB
250ps	350ps	150ps	300ps	200ps

另外，假定处理器中各种指令的使用频度如下：

ALU/Logic	Jump/Branch	Load	Store
45%	20%	20%	15%

4.16.1 [5]<4.6> 流水线处理器与非流水线处理器的时钟周期分别是多少？

4.16.2 [10]<4.6>lw 指令在流水线处理器和非流水线处理器中的总延迟分别是多少？

4.16.3 [10]<4.6> 如果可以将原流水线数据通路的一级拆分为两级，每级的延迟是原级的一半，那么你会选择哪一级进行拆分？拆分后处理器的时钟周期为多少？

4.16.4 [10]<4.6> 假设没有阻塞和冒险，数据存储器的利用率是多少（占总周期数的百分比）？

4.16.5 [10]<4.6> 假设没有阻塞和冒险，寄存器堆的写寄存器端口的利用率是多少？

4.17 [10]<4.6> 在一个具有 *k* 级流水线的 CPU 中，执行 *n* 条指令需要的最少时钟周期数是多少？列出公式并证明。

4.18 [5]<4.6> 假设寄存器 $s0 的初值为 11，$s1 的初值为 22。在 4.6 节所述的不处理数据冒险（即程序员需要在必要的地方插入 NOP 指令来处理数据冒险）的流水线中执行下面的代码，寄存器 $s2 和 $s3 的最终结果分别是多少？

```
addi  $s0, $s1, 5
add   $s2, $s0, $s1
addi  $s3, $s0, 15
```

4.19 [10]<4.6> 假设寄存器 $s0 的初值为 11，$s1 的初值为 22。在 4.6 节所述的不处理数据冒险（即程序员需要在必要的地方插入 NOP 指令来处理数据冒险）的流水线中执行下面的代码，寄存器 $s5 的最终结果分别是多少？假设寄存器堆在时钟周期开始时写，在时钟周期结束时读，那么，ID 级可以返回 WB 级在同一个是时钟周期内产生的结果。4.8 节和图 4-51 有详细的说明。

```
addi  $s0, $s1, 5
add   $s2, $s0, $s1
addi  $s3, $s0, 15
add   $s4, $s0, $s0
```

4.20 [5]<4.6> 在下面的代码中添加 NOP 指令，确保代码能够在无法处理数据冒险的流水线中正确执行。

```
addi  $s0, $s1, 5
add   $s2, $s0, $s1
addi  $s3, $s0, 15
add   $s4, $s2, $s1
```

4.21 考虑 4.6 节所述的无法处理器数据冒险的流水线（即程序员需要在必要的地方插入 NOP 指令来处理器数据冒险）。假设（优化之后）一个典型的 *n* 条指令的程序需要增加 0.4**n* 条 NOP 指令来处理数据冒险。

4.21.1 [5] <4.6> 假设该流水线（没有旁路）的时钟周期是 250 ps，并假设增加旁路硬件可以将 NOP 指令的数量从 0.4**n* 减少到 0.05**n*，但时钟周期会增加到 300 ps。和原先的流水线相比，新流水线的加速比是多少？

4.21.2 [10] <4.6> 不同的程序需要不同数量的 NOP 指令。当一个典型程序运行在带旁路的流水线之前，程序中还用多少 NOP 指令（用指令条数的百分比表示）？

4.21.3 [10]<4.6> 重做练习题 4.21.2，但使用 *x* 代表相对于 *n* 的 NOP 指令的数量（练习题 4.21.2 中，*x* 为 0.4 ），答案用 *x* 表示。

4.21.4 [10] <4.6> 一个使用 0.075**n* 条 NOP 指令的程序能否在带旁路的流水线中执行得更快？请解释原因。

4.21.5 [10] <4.6> 如果想要程序在有旁路得流水线中执行得更快，最少需要有多少 NOP 指令（用指令条数得百分比表示）？

4.22 [5] <4.6> 考虑下面的 MIPS 汇编代码段：

```
sd   $s5, 12($s3)
ld   $s5, 8($s3)
sub  $s4, $s2, $s1
beqz $s4, label
add  $s2, $s0, $s1
sub  $s2, $s6, $s1
```

假设我们可以修改流水线，只保留一个存储器（指令和数据共用一个存储器）。在这种情况下，如果同一个时钟周期内同时取指令和访问数据，将产生结构冒险。

4.22.1 [5] <4.6> 画出上述代码的流水线图，展示是否产生阻塞。

4.22.2 [5] <4.6> 一般来说，重排代码能否减少阻塞或 NOP 指令的数量？

4.22.3 [5] <4.6> 上述结构冒险必须使用硬件处理吗？数据冒险能够通过添加 NOP 指令消除，结构冒险能使用相同的方法处理吗？如果能，给出解决方案；如果不能，请解释原因。

4.22.4 [5]<4.6> 该结构冒险将导致一个典型的程序产生多少阻塞（使用练习题 4.8 中给出的指令组合）？

4.23 如果使用寄存器作为 load/store 指令的存储器地址（无偏移量），那么这些指令将不再需要使用 ALU（参见练习题 4.15 ）。因此，MEM 和 EX 流水级可以重叠，流水线只有 4 级。

4.23.1 [10] <4.6> 流水线深度的减少对时钟周期有何影响？

4.23.2 [5] <4.6> 这种改变将如何提升流水线的性能？

4.23.3 [5] <4.6> 这种改变将如何降低流水线的性能？

4.24 [10] <4.8> 下面的两个流水线示意图中，哪个更好的描述了流水线冒险检测单元的操作？为什么？选择 1：

```
ld x11, 0(x12):     IF ID EX ME WB
add x13, x11, X14:  IF ID EX..ME WB
or x15, x16, X17:   IF ID..EX ME WB
```

选择 2：

```
ld x11, 0(x12):     IF ID EX ME WB
add x13, x11, x14:  IF ID..EX ME WB
or x15, x16, x17:   IF..ID EX ME WB
```

4.25 考虑下面的循环：

```
LOOP: ld   $s0, 0($s3)
      ld   $s1, 8($s3)
      add  $s2, $s0, $s1
      addi $s3, $s3, -16
      bnez $s2, LOOP
```

假设使用了完美的分支预测（即没有控制相关引起的阻塞），没有延迟槽，流水线支持完全旁路机制，分支处理在 EX 级（不是在 ID 级）进行。

4.25.1 [10]<4.8> 画出该循环前两次迭代的流水线执行图。

4.25.2 [10]<4.8> 标记出做无用功的流水级。流水线满负荷工作，某个时钟周期中 5 个流水级全部都在进行有效工作的频率是多少？（从 addi 指令位于 IF 级的时钟周期开始，到 bnez 指令位于 IF 级的时钟周期结束。）

4.26 本练习讨论流水线处理器中旁路的成本 / 复杂度 / 性能之间的折中。参考图 4-43 的流水线数据通路，假设指令中有部分存在 RAW（read after write，写后读）数据相关。RAW 数据相关根据生成结果的流水级（EX 或 MEM）和使用结果的下一条指令（产生结果的指令后紧跟的第 1 条或第 2 条指令）检测。假设在时钟周期的前半部分写寄存器，在后半部分读寄存器，这样“EX 到第 3 条指令”和“MEM 到第 3 条指令”的相关不会产生数据冒险。最后假设无数据冒险时处理器的 CPI 为 1。

EX to 1st Only	MEM to 1st Only	EX to 2nd Only	MEM to 2nd Only	EX to 1st and EX to 2nd
5%	20%	5%	10%	10%

假定各级流水线延迟如下。其中 EX 级给出了不同旁路情况下的延迟。

IF	ID	EX (no FW)	EX (full FW)	EX (FW from EX/MEM only)	EX (FW from MEM/WB only)	MEM	WB
120ps	100ps	110ps	130ps	120ps	120ps	120ps	100ps

4.26.1 [5]<4.8> 对于上面列出的每种 RAW 相关，给出至少 3 条汇编语句来表示该相关。

4.26.2 [5]<4.8> 对于上面列出的每种 RAW 相关，需要在练习题 4.26.1 中你所提供的代码中插入多少 NOP 指令才能在没有旁路机制或冒险检测的流水线中正确执行？给出 NOP 指令插入的位置。

4.26.3 [10]<4.8> 如果在没有旁路机制或冒险检测的流水线中执行程序时，对每条指令独立进行分析，计算需要额外执行的 NOP 指令的数量。写一个 3 条汇编语句组成的代码序列，并单独分析每条指令时，插入的阻塞数将比该序列为了避免数据相关实际所需要的阻塞数多。

4.26.4 [5]<4.8> 假设没有其他冒险，上表中的程序在没有旁路的流水线中运行时，CPU 是多少？（为了简化，假设所有必须的条件都在上表中列出，并都能单独进行分析处理。）

4.26.5 [5]<4.8> 如果采用全旁路机制（旁路转发所有能够转发的结果），CPI 是多少？阻塞的时钟周期所占比例是多少？

CPI

阻塞的时钟周期所占的比例是多少？

4.26.6 [10] <4.8> 假设无法提供全旁路机制所需的三输入多路选择器。现在需要判断只从 EX/MEM 流水线寄存器进行转发（下一时钟周期转发）的效果好，还是只从 MEM/WB 流水线寄存器转发（两个周期后转发）的效果好。每种选择的 CPI 是多少？

4.26.7 [5] <4.8> 对于给出的冒险发生的可能性和各个流水级的延迟，相对于无旁路机制的流水线，每类旁路机制（EX/MEM、MEM/WB 或全旁路）获得的加速比分别是多少？

4.26.8 [5] <4.8> 如果采用“穿越”转发机制来消除所有的数据相关，与练习题 4.26.7 提供的最快处理器相比，此时获得的额外的加速比是多少？假设这种带实现的“穿越”转发机制使得全转发的 EX 级的延迟增加 100 ps。

4.26.9 [5] <4.8> 冒险类型表区分了“ EX to 1st”以及“ EX to 1st and EX to 2nd”的入口。为什么没有“MEM to 1st and MEM to 2nd”的入口？

4.27 本题用到下面的指令序列。假设该指令序列在一个 5 级流水线的数据通路中执行：

```
add  $s3, $s1, $s0
lw   $s2, 4($s3)
lw   $s1, 0($s4)
or   $s2, $s3, $s2
sw   $s2, 0($s3)
```

排

4.27.1 [5] <4.8> 如果没有旁路或者冒险检测，插入 NOP 指令以保证该序列能够正确执行。

4.27.2 [10] <4.8> 现在，修改或重排该指令序列，来将需要的 NOP 指令减到最少。假设在修改后的代码中，寄存器 $t0 用于存储临时变量。

4.27.3 [10] <4.8> 假设处理器有旁路机制，但忘记了实现冒险检测单元，那么题目提供的原始代码在执行时会发生什么情况？

4.27.4 [10] <4.8> 假设有旁路机制，在代码执行的前 7 个时钟周期，图 4-59 中的旁路单元和冒险检测单元在每个时钟周期将发出哪些信号？

4.27.5 [10] <4.8> 如果没有旁路机制，图 4-59 中的冒险检测单元需要哪些新的输入和输出信号？用本题提供的指令作为例子，解释每个信号增加的原因。

4.27.6 [20] <4.8> 对于练习题 4.26.5 提供的新的冒险检测单元，说明指令序列执行的前 5 个时钟周期都发出了哪些输出信号。

4.28 一个好的分支预测器的作用取决于条件分支执行的频率，它与分支预测器的精度共同决定误预测导致的阻塞时间长短。在本练习中，假设指令的动态执行频度如下：

R-type	beqz/bnez	jal	lw	sw
40%	25%	5%	25%	5%

假定分支预测器的精度如下：

分支总发生	分支总不发生	2位预测器
45%	55%	85%

4.28.1 [10] <4.9> 误预测导致的阻塞将增加 CPI。对分支总发生预测器而言，误预测将导致 CPI 增加多少？假设没有数据冒险且不使用延迟槽，分支结果在 ID 级产生，在 EX 级使用。

4.28.2 [10] <4.9> 重做练习题 4.28.1，这次改为使用分支总不发生预测器。

4.28.3 [10] <4.9> 重做练习题 4.28.1，这次改为使用 2 位分支预测器。

4.28.4 [10] <4.9> 对 2 位分支预测器而言，将一半分支指令用 ALU 指令替代（一条 ALU 指令替代一条分支指令）将获得的加速比是多少？假设被正确预测的分支指令和被不正确预测的分支

指令被取代的概率相同。

4.28.5 [10] <4.9> 对 2 位分支预测器而言，将一半分支指令用 ALU 指令替代（两条 ALU 指令替代一条分支指令）将获得的加速比是多少？假设被正确预测的分支指令和被不正确预测的分支指令被取代的概率相同。

4.28.6 [10] <4.9> 有些分支是非常容易预测的。假设 80% 的分支指令都是非常容易预测的循环回环（loop-back）分支，那么 2 位分支预测器对剩下的 20% 分支指令的预测精度是多少？

4.29 本练习讨论不同分支预测器对给定分支模式（如循环）的预测精度。给定的分支模式为：T,NT,T,T,NT。

4.29.1 [5] <4.9> 对该分支模式，分支总发生预测器与分支总不发生预测器的准确率分别是多少？

4.29.2 [5] <4.9> 对该分支模式的前 4 个分支而言，2 位分支预测器的准确率是多少？假设预测器的初始状态与图 4-61 左下角状态相同（预测不发生）。

4.29.3 [10] <4.9> 如果该分支模式一直重复下去，2 位分支预测器的准确率是多少？

4.29.4 [30] <4.9> 如果该分支模式一直重复下去，设计一个能取得最高准确率的预测器。这个预测器必须是一个时序电路，有一个输出表示预测结果（1 表示发生，0 表示未发生），除了时钟和指示当前指令是条件分支指令的信号外没有其他输入。

4.29.5 [10] <4.9> 如果有一个分支模式与该分支模式完全相反且一直重复下去，那么在练习题 4.29.4 中设计的预测器对这个分支的准确率是多少？

4.29.6 [20] <4.9> 重做练习题 4.29.4，这次设计的预测器最终（可能需要一个热身过程）可以同时完美地预测该分支模式及完全相反的分支模式（假设分支模式一直重复下去）。这个预测器应该有一个输入告诉它真实的分支结果。提示：这个输入可以帮助预测器判断是两个分支模式中的哪一个。

4.30 本练习题讨论异常处理对流水线设计的影响。根据下面两条指令回答前三个问题：

指令1	指令2
beqz $s0, LABEL	ld $s0, 0($s1)

4.30.1 [5] <4.10> 每条指令分别可能产生什么异常？对每个可能产生的异常，指出其将在哪个流水线级被检测到。

4.30.2 [10] <4.10> 如果每个异常都有独立的异常处理程序入口地址，流水线应该怎样设计才能处理异常？假设设计处理器时已知每个异常处理程序入口地址。

4.30.3 [10] <4.10> 如果第二条指令紧跟第一条指令之后立即被取出，试说明第一条指令发生异常（见练习题 4.30.1）时流水线的运行情况。给出从第一条指令取指开始到异常处理程序第一条指令完成时的流水线运行图。

4.30.4 [20] <4.10> 在向量化异常处理中，异常处理程序的入口地址表存放在数据存储器的一个固定位置。改变流水线的实现以支持向量化异常处理。重做练习 4.30.3，这次使用支持向量化异常处理的流水线。

4.30.5 [15] <4.10> 我们想要在仅有一个固定处理程序地址的处理器上模拟向量异常处理（见练习题 4.30.4），写出相应的程序。提示：这段程序应识别异常类型，从异常向量表中获得正确地址，然后跳转到该异常处理程序处。

4.31 本练习比较单发射和双发射处理器的性能，并考虑对双发射处理器进行程序优化。本题使用下面的循环（C 语言编写）：

```
for(i=0;i!=j;i+=2)
   b[i]=a[i]-a[i+1];
```

进行少量优化或不进行优化的编译器可能产生下面这段 MIPS 汇编代码：

上面的代码用到下面这些寄存器：

```
     li   $s0, 0
     jal  ENT
TOP: sll  $t0, $s0, 3
     add  $t1, $s2, $t0
     lw   $t2, 0($t1)
     lw   $t3, 8($t1)
     sub  $t4, $t2, $t3
     add  $t5, $s3, $t0
     [illegible], 0($t5)
     [illegible], $s0, 2
ENT: bne  $s0, $s1, TOP
```

假设本题中的双发射、静态调度处理器具有如下特性：

1. 一个指令必须是存储器操作，另一条指令必须是算术 / 逻辑运算指令或一个分支指令。
2. 处理器流水线的各级之间有所有可能的旁路路径（包括到 ID 级的用于解决分支问题的路径）。
3. 处理器有完美的分支预测。
4. 如果一条指令与另外一条指令相关，两条指令不会在同一个包中一起发射（见 4.11.2 节）。
5. 如果需要阻塞，发射包中的两条指令都需要阻塞（见 4.11.2 节）。

做完这些练习你会发现，要获得达到近似最优加速比的代码需要做多少工作。

4.31.1 ［30］<4.11> 画出一个流水线执行图，展示上述 MIPS 代码在双发射处理器上如何执行。假设两个迭代后循环退出。

4.31.2 ［10］<4.11> 从单发射处理器到双发射处理器的加速比是多少？（假设循环迭代数千次。）

4.31.3 ［10］<4.11> 重排或重写上述 MIPS 代码，以便在双发射处理器上获得更好的性能。提示：如果 j=0，使用指令 beqz $s1，DONE 直接跳出循环。

4.31.4 ［20］<4.11> 重排或重写上述 MIPS 代码，以便在双发射处理器上获得更好的性能，但不要展开循环。

4.31.5 ［30］<4.11> 重复练习题 4.31.1，此次使用练习题 4.31.4 中的优化代码。

4.31.6 ［10］<4.11> 执行练习题 4.31.3 和练习题 4.31.4 中的优化代码，从单发射处理器到双发射处理器的加速比是多少？

4.31.7 ［10］<4.11> 将练习题 4.31.3 中的 MIPS 代码进行循环展开，展开的循环中每个迭代可以处理器原代码中的两个迭代。然后，重排或重写展开的代码，以便在单发射处理器上获得更好的性能。假设 j 是 4 的倍数。

4.31.8 ［20］<4.11> 将练习题 4.31.4 中的 MIPS 代码进行循环展开，展开的循环中每个迭代可以处理器原代码中的两个迭代。然后，重排或重写展开的代码，以便在双发射处理器上获得更好的性能。假设 j 是 4 的倍数。（提示：重新组织循环，使得一些计算出现在循环体的外面和循环结束处。可以假设临时寄存器中的值在循环之后不再需要。）

4.31.9 ［10］<4.11> 执行练习题 4.31.7 和练习题 4.31.8 中循环展开以及优化后的代码，从单理器到双发射处理器的加速比是多少？

4.31.10 ［30］<4.11> 重复练习题 4.31.8 和练习题 4.31.9，但此次假设双发射处理器能同时执行两个算术 / 逻辑指令。（也就是说，发射包中的第一条指令可以是任意类型的指令，但第二条指令必须是算术或逻辑指令。两条存储器操作指令不能同时被调度。）

4.32 本题讨论能效与性能的关系。假设数据通路各部件（指令存储器、寄存器、数据存储器）的

活动功耗如下表所示，其他部件的功耗可以忽略（寄存器读和寄存器写只针对寄存器堆）。

I-Mem	1 Register Read	Register Write	D-Mem Read	D-Mem Write
140pJ	70pJ	60pJ	140pJ	120pJ

假定数据通路上的部件延迟如下表所示，其他功能部件的延迟可以忽略。

I-Mem	Control	Register Read or Write	ALU	D-Mem Read or Write
200ps	150ps	90ps	90ps	250ps

4.32.1 [5] <4.3,4.7,4.15> 在单周期处理器和 5 级流水线处理器中执行一条加法指令，分别消耗多少能量？

4.32.2 [10] <4.7,4.15> 消耗能量最大的 MIPS 指令是哪一条？执行这条指令的能耗是多少？

4.32.3 [10] <4.7,4.15> 如果降低能耗是最重要的，应该怎样修改流水线设计？改进之后，执行一条 lw 指令时的能耗降低的比例是多少？

4.32.4 [10] <4.7,4.15> 还有哪些指令能从练习题 4.32.3 的改进中潜在收益？

4.32.5 [10] <4.7,4.15> 练习题 4.32.3 的改进如何影响 CPU 的性能？

4.32.6 [10] <4.7,4.15> 我们可以去掉 MemRead 控制信号，即每个周期都读数据存储器（MemRead 恒为 1）。解释为什么去掉该控制信号后处理器依然能正常工作。它对时钟频率和能耗又有什么影响？

4.33 在制造硅芯片时，材料（例如，硅）的缺陷和制造错误会导致电路失效。一个非常普遍的问题是一根线上的信号会对相邻线上的信号产生影响，这被称为串扰。有一类串扰问题是这样的，某些线上的信号为常值（如电源线），该线附近的线也被固定为 0（stuck-at-0）或 1（stuck-at-1）。下面的问题中的缺陷发生在图 4-21 中寄存器堆的写寄存器输入端的第 0 位。

4.33.1 [10] <4.3，4.4> 假设这样测试处理器的缺陷：（1）先给 PC、寄存器堆、数据和指令存储器中设置一些值（可以自己选择），（2）执行一条指令，（3）然后读出 PC、寄存器堆和存储器中的值；最后检查这些值以判断处理器中是否存在缺陷。能否设计一个测试（PC、存储器以及寄存器的值）方案，检查该信号上是否有固定为 0 缺陷？

4.33.2 [10] <4.3,4.4> 重做练习题 4.33.1，这次检查固定为 1 缺陷。你能只设计一个测试方案同时检查固定为 0 缺陷和固定为 1 缺陷吗？如果可以，请解释如何实现；如果不能，请说明理由。

4.33.3 [10] <4.3,4.4> 如果我们知道一个处理器在该信号上有一个固定为 1 缺陷，那么这个处理器还能用吗？为了使这个处理器仍然可用，我们必须将原来能在正常 MIPS 处理器上运行的程序作一些变换，使之可以在这个处理器上运行。假设指令存储器和数据存储器都很大，足够容纳变换后的程序。

4.33.4 [10] <4.3,4.4> 重做练习题 4.33.1，此次检测的错误是，Branch 控制信号为 0 时，MemRead 控制信号是否变为 0，变化则有缺陷，否则无缺陷。

4.33.5 [10] <4.3,4.4> 重做练习题 4.33.1，此次检测的错误是，RegDst 控制信号为 1 时，MemRead 信号是否变为 1，变化则有缺陷，否则无缺陷。提示：该题需要操作系统的知识，考虑一下什么会引起段错误（segmentation fault）。

小测验答案

4.1 5 个中的 3 个：控制器、数据通路、存储器。省略了输入和输出。

4.2 错。边沿触发状态单元可以同时进行读写。

4.3 I.a；II.c。

4.4　是，Branch 与 ALUOp0 是相同的。而且，MemtoReg 和 RegDst 是相反的，不需要额外的反相器。仅使用另外一个信号，并翻转多路选择器的输入即可。

4.5　1. 错误。2. 可能；如果信号 PCSource[0] 一直为 0，当它是无关项时（大多数情况），与 PCWriteCond 相同

4.6　1. 因为 `lw` 的结果而阻塞；2. 旁路第一个 `add` 的结果写入 `$t1`；3. 不需要阻塞或旁路。

4.7　2 和 4 正确，其余错误。

4.9　1. 预测不发生；2. 预测发生；3. 动态预测。

4.10　第一条指令，因为在逻辑上它最先执行。

4.11　1. 都有；2. 都有；3. 软件；4. 硬件；5. 硬件；6. 硬件；7. 都有；8. 硬件；9. 都有。

4.13　前两个错误，后两个正确。

HZ BOOKS
华章图书